claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_f9255361-69d\agent-a65f416985d7bbe99.jsonl`
- SHA-256: `d06c231561f406e9d61af25b06f4151091e073aead8727d98b3cc6db208aa265`
- Source modified: `2026-06-22T09:34:48+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_f9255361-69d`
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`

Transcript

1. user (2026-06-22T09:32:46.029Z)

You are auditing ONE doc in the KhelSutra brand/site repo for staleness, as part of a "are all the docs up to date?" sweep.

Repo root: C:\Users\avidu\Projects\khelsutra (bash path: /c/Users/avidu/Projects/khelsutra)
The repo is on `master`, freshly synced to remote HEAD ? the latest merged PR is #94. Today is 2026-06-22.

DOC TO AUDIT: docs/PORTABILITY.md

Your job: read the doc IN FULL, extract every concrete, falsifiable claim, and VERIFY each against the LIVE repo ? do NOT trust the doc's own wording. Hard discipline rule: a claim is "correct" ONLY when you have file:line or command-output evidence for it; if you cannot verify it, mark verdict "unverifiable" ? never guess, never assume the doc is right. Re-check against actual code/git.

Kinds of claims to check:
- File / path / route references ("served at /capture", "see src/store.js", "public/og.png", "systemd khelsutra-site") -> does the path or route ACTUALLY exist now? Confirm with git ls-files / Read / the routing in site/server.js + site/src/index.js.
- PR numbers and counts ("PRs #1-#8 merged", "20 tests green", "6 languages", "3-step wizard", "P0..P5 done") -> verify against `git -C <root> log`, the actual test output/count, and the code.
- Status markers (DONE / pending / IN PROGRESS / DRAFT / "?" / "owner-gated" / "NEXT") -> still accurate vs the code & git history? Is something marked pending that is actually shipped, or vice-versa?
- Dates, "live at <url>", "merged in #N".
- Whether the doc describes something now SUPERSEDED or fully shipped -> is it an archive candidate (a tracker whose work is complete, or content duplicated by docs/archives/)? Note: docs/archives/FRIENDLY_AND_MULTILINGUAL.md already exists as the archived multilingual tracker.

Tools: Read the doc + every cited file; Grep/Glob to locate code; Bash for `git -C /c/Users/avidu/Projects/khelsutra log --oneline`, `... ls-files`, and greps. To check a test count, you may run the suite (`cd /c/Users/avidu/Projects/khelsutra/site && node --test` or `npm test`), but prefer cheap checks. Verify against master as it stands NOW.

Severity: high = a factually wrong claim a reader would act on (wrong path/route, a "DONE" that isn't, a wrong count/PR that misleads). medium = outdated-but-harmless (stale PR range, old "where we are", superseded framing). low = cosmetic.

Do NOT modify any file ? audit only. Return ONLY the structured object (doc, overallStatus, findings[], summary).

2. assistant (2026-06-22T09:32:49.403Z)

I'll audit `docs/PORTABILITY.md` in the KhelSutra repo. Let me start by reading the doc in full and syncing context about the repo.

3. user (2026-06-22T09:32:49.889Z)

```
1       # Portability ? Cloudflare now, always-on VM if needed
2
3       > **Status:** design + proven (2026-06-20). **Decision:** keep the site/form host-portable so
4       > moving off Cloudflare to an owner-run always-on VM ("if Cloudflare acts out") is an **adapter
5       > swap, not a rewrite.**
6
7       ## The shape that makes it portable
8       - **Logic is runtime-agnostic.** `src/index.js` uses Web-standard `Request`/`Response` and pure
9         helpers (`validate`, `toRow`, `isBot`, `handleRequest`) ? no host-specific calls.
10      - **All DB coupling is in adapters** (`src/store.js`): `d1Store` (Cloudflare D1) and `sqliteStore`
11        (node:sqlite / better-sqlite3). `handleRequest(request, store)` takes the store by injection.
12      - **One schema, one SQL.** `schema.sql` and the `INSERT` are plain SQLite, so D1 and a VM's SQLite
13        file are byte-for-byte compatible ? zero dialect change. (Postgres later = minor placeholder tweaks.)
14
15      ## Two hosts, same code
16      | | Cloudflare (now) | Always-on VM (fallback) |
17      |---|---|---|
18      | Entry | `src/index.js` (Worker `fetch`) | `server.js` (`node:http`) |
19      | Static | `env.ASSETS` | `server.js` serves `public/` |
20      | Storage | `d1Store(env.DB)` ? D1 | `sqliteStore(db)` ? `node:sqlite` file |
21      | Country | `cf-ipcountry` header | optional (GeoIP / null) |
22      | Deploy | git ? Workers Builds | `node server.js` under systemd |
23      | TLS/DNS | Cloudflare | keep Cloudflare in front, **or** Caddy/Let's Encrypt |
24
25      **Enforced, not asserted.** `node --test src/` runs BOTH paths and CI (`.github/workflows/test.yml`,
26      Node 24) runs them on every PR, so the repo stays compatible with both hosts:
27      - `src/index.test.js` ? the Cloudflare Worker + D1 logic.
28      - `src/server.test.js` ? **boots the real `server.js`** (node:http + node:sqlite) and round-trips it
29        over HTTP into SQLite (the same proof we run on the droplet, automated).
30
31      For a live check against a real VM: `SSH_TARGET=root@host scripts/droplet-smoke.sh` (scp + boot +
32      round-trip + cleanup). Verified live on the owner's droplet (Ubuntu 24.04, isolated `/srv/khelsutra-site`).
33
34      ## Provider-portable provisioning ? destroy ? resume anywhere (scripted + TESTED)
35      Spinning up a fresh VM on **any** provider (DigitalOcean / GCE / any Ubuntu box) and tearing it down
36      without losing data is a **first-class, scripted** flow ? so destroying a box and resuming elsewhere is
37      routine, not risky.
38
39      - **`scripts/provision-vm.sh`** ? run from your LOCAL machine against a FRESH Ubuntu VM. Pushes `site/`
40        via `git archive` (**private-repo-safe** ? no clone, token, or secret on the VM), installs Node (?24,
41        so `node:sqlite` is unflagged), installs + starts a **systemd** service, opens the firewall, and
42        **verifies** it serves (form + a real POST round-trip). Idempotent.
43        ```
44        SSH_TARGET=root@<ip> [SSH_KEY=~/.ssh/khelsutra_droplet] [PORT=8787]
```

```
[RESTORE_DB=./backup.db] \
  site/scripts/provision-vm.sh
```
- **`scripts/teardown-vm.sh`** ? **exports the SQLite state DB to a local timestamped backup BEFORE**
  anything is destroyed (registrations survive the move), stops the service, and optionally destroys the
  DO droplet (`DESTROY=1 DROPLET_ID=<id>`, via `doctl`).
  ```
  SSH_TARGET=root@<ip> [BACKUP_DIR=./vm-state-backups] [DESTROY=1 DROPLET_ID=<id>] \
    site/scripts/teardown-vm.sh
  ```
- **Clean resume on a different provider:** `teardown-vm.sh` (export) ? destroy ? `RESTORE_DB=<backup>`
  SSH_TARGET=root@<newip> site/scripts/provision-vm.sh`. The restored DB is `scp`'d in before the service starts.

Verified live (2026-06-20): a fresh DO Ubuntu-24.04 droplet went bare ? operational + publicly
serving (HTTP 200) via `provision-vm.sh` (Node 24 auto-installed, systemd unit, firewall); `teardown-vm.sh`
exported the state DB (12 KB); a re-provision with `RESTORE_DB` preserved the registration (`rows:2` =
restored + the verify POST) ? then the throwaway droplet was destroyed. The destroy?resume cycle is
reproducible across providers.

VM deploy runbook ("if Cloudflare acts out")
Node 22+ only ? **no `npm install`** (node:sqlite is built in).
```
# on the VM:
git clone ? && cd khelsutra/site        # or scp the site/ folder
PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
# put it under systemd (auto-restart) + a reverse proxy (Caddy/nginx) for TLS,
# then point DNS at the VM. ~1 hour; logic + schema unchanged.
```

Honest notes
- "Cloudflare acts out" usually means **Workers/D1 pricing or limits, not DNS** ? so the cheapest
  move is to keep Cloudflare DNS/CDN in front and shift only the **compute** to the VM (point a
  route/origin at the box). Full exit (own DNS + Caddy TLS) is also supported.
- You can run **both** (Cloudflare primary + VM warm standby) since the schema is identical; a
  periodic `access_requests` export keeps them in sync.
- `server.js` is intentionally dependency-free; if you later want Postgres or a framework (Hono runs
  on Workers *and* Node), it slots in behind the same `store` + handler.

4. user (2026-06-22T09:32:50.879Z)

```
50b2bd7 chore(repo): ignore local runtime artifacts + correct stale NEXT_STEPS claims (#94)
e2570e4 docs: archive the Friendly & Multilingual tracker (engineering complete) (#93)
0bef61e docs(friendly-multilingual): close out the chapter ? log #89/#90/#91 + strengthen fitNav tests (#92)
9f77ddc test(site): browser layout-regression CI guard + auto-collapse the nav to fit every locale (#91)
4415fe8 feat(site): localize remaining home sections (6 langs) + smooth the responsive header (#90)
26c9da1 feat(site): localize the hero demo mockup + #how feature block (6 languages) (#89)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages) (#88)
cf59fbf feat(site): add es/da/id to the marketing site (parity with the SPA's 6 languages) (#87)
```

```
17242a9 feat(site): localize own-model body content to kn/hi (#86)
7b04f02 deploy(kit): align with the live-verified droplet deploy (+ the gemini-default fix)
(#85)
5e5a2ac docs(friendly-multilingual): close out the chapter (§9 DoD + UI/UX + B7 deferred) (#84)
4c0f35c fix(site): UI/UX ? compact language switcher + header overflow fix + localized hero
(#83)
0fae364 feat(web+site): A2 ? localized WhatsApp share for the reel (SPA + site) (#82)
1e52cfd deploy(droplet): tailored alpha kit for the CPU droplet (engine api. side) (#81)
64076a5 test(web): add a coverage floor (CI-gated) + i18n regression tests for the #77 audit
(#80)
8328721 docs(tracker): refresh the §7 milestone prose to merged reality (A6/A7/#77 landed) (#79)
3f0cdb8 docs(deploy): fix two would-break-the-deploy gaps in the alpha runbook
(RALLY_ALLOWED_HOSTS + gs:// jail) (#78)
9f1ec9c fix(i18n): native-review pass on the kn/hi SPA console catalogs (#77)
9a2d7d2 feat(site): A7 ? camera-placement diagram on the capture page (#76)
aa8d5e9 feat(site): A6 ? plain-language Request-Access GPU/storage questions (#75)
```

5. assistant (2026-06-22T09:32:54.402Z)

The doc is structured. Now let me verify the claims. The doc references files in `site/` (the table and provisioning sections reference `site/scripts/...` and `server.js`, `src/index.js`, `src/store.js`, etc.). Let me check the actual repo layout and the cited files.

6. user (2026-06-22T09:32:56.554Z)

```
site/schema.sql
site/scripts/droplet-smoke.sh
site/scripts/provision-vm.sh
site/scripts/teardown-vm.sh
site/server.js
site/src/assets.test.js
site/src/deploy-config.test.js
site/src/i18n.shim.test.js
site/src/index.js
site/src/index.test.js
site/src/locales-drift.test.js
site/src/server.test.js
site/src/store.js
site/src/ux.dom.test.js
```

7. user (2026-06-22T09:32:58.024Z)

```
.github/workflows/parity.yml
.github/workflows/test.yml
site/scripts/copy-locales.mjs
site/scripts/droplet-smoke.sh
site/scripts/parity-check.sh
site/scripts/provision-vm.sh
site/scripts/teardown-vm.sh
```

8. assistant (2026-06-22T09:33:02.082Z)

All cited files exist. Now let me verify the specific claims: the helper functions in `src/index.js`, the stores in `src/store.js`, the entry/server in `server.js`, the CI Node version, and the test files' content.

9. user (2026-06-22T09:33:02.997Z)

```
1       // Storage adapters for Request-Access submissions.
2       //
3       // PORTABILITY (see docs/PORTABILITY.md): the business logic (validate / toRow /
4       // handleRequest in index.js) is runtime-agnostic ? ALL database coupling lives here.
5       // The Cloudflare Worker uses d1Store(env.DB); an always-on VM uses sqliteStore(db)
6       // over node:sqlite (or better-sqlite3). Same schema.sql, same INSERT ? zero dialect
```

```
7       // change. To move hosts, swap the adapter, not the logic.
8
9       export const INSERT_SQL =
10        `INSERT INTO access_requests
11           (segment, name, email, city, org_name, courts, has_gpu, storage, message, consent,
country, user_agent)
12         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`;
13
14      export const insertArgs = (r) => [
15        r.segment, r.name, r.email, r.city, r.org_name, r.courts,
16        r.has_gpu, r.storage, r.message, r.consent, r.country, r.user_agent,
17      ];
18
19      // Cloudflare D1 (async): prepare(sql).bind(...args).run()
20      export function d1Store(DB) {
21        return {
22          async insertAccessRequest(r) {
23            await DB.prepare(INSERT_SQL).bind(...insertArgs(r)).run();
24          },
25        };
26      }
27
28      // node:sqlite / better-sqlite3 (sync): prepare(sql).run(...args). For the VM host.
29      export function sqliteStore(db) {
30        const stmt = db.prepare(INSERT_SQL);
31        return {
32          async insertAccessRequest(r) {
33            stmt.run(...insertArgs(r));
34          },
35        };
36      }
37
```

10. user (2026-06-22T09:33:03.233Z)

```
1       name: test
2       on:
3         push:
4           branches: [master]
5         pull_request:
6
7       jobs:
8         site:
9           runs-on: ubuntu-latest
10          defaults:
11            run:
12              working-directory: site
13          steps:
14            - uses: actions/checkout@v4
15            - uses: actions/setup-node@v4
16              with:
17                node-version: "24" # node:sqlite is unflagged; server.test.js boots the VM
host
18                cache: "npm"
19                cache-dependency-path: site/package-lock.json
20            # jsdom is a TEST-ONLY devDep (the behavioral landing-page UX tests run the real
inline
21            # script in a DOM); the Cloudflare Worker + VM-host RUNTIMES stay dependency-free.
`npm test`
22            # runs ALL paths: Worker+D1 logic (index.test.js), the booted VM host
(server.test.js), the
23            # static content contract (assets.test.js), and the UX behaviour (ux.dom.test.js).
24            - run: npm ci
25            - run: npm test
26            # Syntax-check the ops shell scripts (provision/teardown/smoke/parity) so a broken
```

```
27            # script can't ship to the runbook. `bash -n` parses without executing.
28            - name: shell scripts parse-check
29              run: for f in scripts/*.sh; do echo "checking $f"; bash -n "$f"; done
30
31        web:
32          # Operator-console SPA (web/): typecheck + unit tests (with a coverage floor) +
production build
33          # gate the merge. `test:coverage` fails if coverage drops below the thresholds in
vite.config.ts,
34          # so a merge can't silently regress test coverage.
35          runs-on: ubuntu-latest
36          defaults:
37            run:
38              working-directory: web
39          steps:
40            - uses: actions/checkout@v4
41            - uses: actions/setup-node@v4
42              with:
43                node-version: "24"
44                cache: "npm"
45                cache-dependency-path: web/package-lock.json
46            - run: npm ci
47            - run: npm run typecheck
48            - run: npm run test:coverage
49            - run: npm run build
50
51        layout:
52          # Real-browser layout-regression guard. The other suites run in jsdom, which has NO
layout engine
53          # (getBoundingClientRect() is all zeros) and therefore cannot see header overflow /
wrapping /
54          # clipping ? the responsive-nav bug that recurred. This drives a headless Chromium
over the real
55          # pages at a spread of viewport widths x all 6 locales and asserts the header never
distorts.
56          runs-on: ubuntu-latest
57          defaults:
58            run:
59              working-directory: site
60          steps:
61            - uses: actions/checkout@v4
62            - uses: actions/setup-node@v4
63              with:
64                node-version: "24"
65                cache: "npm"
66                cache-dependency-path: site/package-lock.json
67            - run: npm ci
68            - run: npx playwright install --with-deps chromium
69            - run: npm run test:layout
70
```

11. assistant (2026-06-22T09:33:07.072Z)

`store.js` confirms `d1Store`, `sqliteStore`, `INSERT_SQL`. CI uses Node 24. Note: the doc says "`node --test src/` runs BOTH paths" ? but CI actually runs `npm test`. Let me check what `npm test` runs and verify `src/index.js` helpers, `server.js`, and `schema.sql`.

12. user (2026-06-22T09:33:08.180Z)

```
1       // Cloudflare Worker for khelsutra.guru ? serves the static site and handles the
2       // Request-Access form: POST /api/request-access -> validate -> insert into D1 (binding
DB).
3       // Pure helpers are exported so they can be unit-tested without a live Worker/D1.
4
5       import { d1Store } from "./store.js";
```

```
import { cfEmailNotify } from "./notify.js";

const SEGMENTS = ["academy", "individual"];
const GPU = ["yes", "unsure", "no", "prefer-cloud"];
const COURTS = ["1", "2-4", "5+"];
const STORAGE = ["gdrive", "onedrive", "dropbox", "bucket", "local", "unsure"];

function str(v, max) {
  if (v == null) return null;
  const s = String(v).trim();
  return s ? s.slice(0, max) : null;
}

// Honeypot: humans never fill the hidden "company" field.
export function isBot(p) {
  return !!(p && typeof p.company === "string" && p.company.trim() !== "");
}

export function validate(p) {
  if (!p || typeof p !== "object") return { ok: false, error: "bad payload" };
  const email = String(p.email || "").trim();
  if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(email)) return { ok: false, error: "valid email
required" };
  return { ok: true };
}

export function toRow(p, meta = {}) {
  const storage = Array.isArray(p.storage)
    ? p.storage.filter((s) => STORAGE.includes(s)).join(",")
    : "";
  return {
    segment: SEGMENTS.includes(p.segment) ? p.segment : null,
    name: str(p.name, 200),
    email: str(p.email, 320),
    city: str(p.city, 200),
    org_name: str(p.org_name, 200),
    courts: COURTS.includes(p.courts) ? p.courts : null,
    has_gpu: GPU.includes(p.has_gpu) ? p.has_gpu : null,
    storage,
    message: str(p.message, 4000),
    consent: p.consent ? 1 : 0,
    country: str(meta.country, 8),
    user_agent: str(meta.userAgent, 400),
  };
}

function json(obj, status = 200) {
  return new Response(JSON.stringify(obj), { status, headers: { "content-type":
"application/json" } });
}

export async function handleRequest(request, store, notify) {
  let body;
  try {
    body = await request.json();
  } catch {
    return json({ ok: false, error: "invalid json" }, 400);
  }
  if (isBot(body)) return json({ ok: true }); // silently drop bots
  const v = validate(body);
  if (!v.ok) return json({ ok: false, error: v.error }, 400);

  const r = toRow(body, {
    country: request.headers.get("cf-ipcountry"),
    userAgent: request.headers.get("user-agent"),
```

```
69        });
70
71        await store.insertAccessRequest(r);
72        try { await notify?.notify(r); } catch { /* email is best-effort; never fail the
request */ }
73
74        return json({ ok: true });
75      }
76
77      export default {
78        async fetch(request, env) {
79          const url = new URL(request.url);
80          if (url.pathname === "/api/request-access") {
81            if (request.method !== "POST") return json({ ok: false, error: "method not
allowed" }, 405);
82            return handleRequest(request, d1Store(env.DB), cfEmailNotify(env.NOTIFY,
env.NOTIFY_FROM, env.NOTIFY_TO));
83          }
84          return env.ASSETS.fetch(request); // everything else: static assets
85        },
86      };
87
```

13. user (2026-06-22T09:33:08.897Z)

```
  "scripts": {
    "test": "node --test",
    "test:cf": "node --test src/index.test.js",
    "test:vm": "node --test src/server.test.js",
    "test:deploy": "node --test src/deploy-config.test.js src/assets.test.js",
    "test:layout": "node --test src/layout.browser.mjs",
    "copy:locales": "node scripts/copy-locales.mjs",
    "predeploy": "node scripts/copy-locales.mjs",
    "deploy": "wrangler deploy",
    "dev": "wrangler dev",
    "og": "python make_og.py"
  },
  "devDependencies": {
    "jsdom": "^29.1.1",
    "playwright": "^1.61.0"
  }
```

14. assistant (2026-06-22T09:33:13.684Z)

The helpers `validate`, `toRow`, `isBot`, `handleRequest` all exist in `index.js`. The doc claims `cf-ipcountry` header is used ? confirmed at line 67. Note: the doc's claim "`node --test src/` runs BOTH paths" ? the actual test command is `node --test` (no `src/` arg) and `test:cf`/`test:vm` are the split targets. This is a minor inaccuracy in the command string. Let me verify `server.js` (entry, serves `public/`, node:sqlite, PORT/DB_PATH env), `schema.sql`, and the test files boot the real server.

15. user (2026-06-22T09:33:18.505Z)

```
1       // Always-on VM host for the KhelSutra site ? dependency-free (Node 22+ only).
2       // Serves public/ and POST /api/request-access into SQLite (node:sqlite), using the
3       // SAME handler logic as the Cloudflare Worker (src/index.js). This is the portability
4       // fallback "if Cloudflare acts out" ? see docs/PORTABILITY.md.
5       //
6       //   PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
7       //
8       import { createServer } from "node:http";
9       import { readFile } from "node:fs/promises";
10      import { join, extname, normalize } from "node:path";
11      import { DatabaseSync } from "node:sqlite";
```

```
12      import { handleRequest } from "./src/index.js";
13      import { sqliteStore } from "./src/store.js";
14      import { noopNotify } from "./src/notify.js";
15
16      const PORT = Number(process.env.PORT) || 8080;
17      const ROOT = import.meta.dirname;
18      const PUBLIC = join(ROOT, "public");
19      const DB_PATH = process.env.DB_PATH || join(ROOT, "access.db");
20
21      const db = new DatabaseSync(DB_PATH);
22      db.exec(await readFile(join(ROOT, "schema.sql"), "utf8")); // CREATE TABLE IF NOT EXISTS
? idempotent
23      const store = sqliteStore(db);
24      const notify = noopNotify(); // VM host: no email yet (add SMTP later ? see
docs/PORTABILITY.md)
25
26      const TYPES = {
27        ".html": "text/html; charset=utf-8", ".png": "image/png", ".svg": "image/svg+xml",
28        ".css": "text/css", ".js": "text/javascript", ".json": "application/json;
charset=utf-8",
29        ".ico": "image/x-icon", ".txt": "text/plain",
30        ".mp4": "video/mp4", ".webm": "video/webm", ".gif": "image/gif", ".jpg": "image/jpeg",
".jpeg": "image/jpeg",
31      };
32
33      // --- Language negotiation (tracker B3 / §6.5 N8)
------------------------------------------------
34      // The trilingual marketing site. The client shim (public/i18n.js) does the full
negotiation in the
35      // browser; this VM host PRE-RESOLVES the locale server-side so the first paint (and any
no-JS read)
36      // already carries the right <html lang>. Same ORDER as the SPA + shim: persisted pref
-> ?lang= ->
37      // navigator/Accept-Language -> en. (On Cloudflare, assets are served directly and only
the client
38      // shim negotiates ? see src/index.js.)
39      const SITE_LOCALES = ["en", "kn", "hi", "es", "da", "id"];
40      const LANG_COOKIE = "khelsutra.lang";
41      const LOCALIZED_PAGES = new Set(["index.html", "capture.html", "own-model.html"]);
42
43      const baseLang = (code) => (code ?
String(code).toLowerCase().split(";")[0].trim().split("-")[0] : "");
44
45      function cookieValue(header, name) {
46        const m = (header || "").match(new RegExp("(?:^|;\\s*)" + name.replace(/\./g, "\\.") +
"=([^;]*)"));
47        return m ? decodeURIComponent(m[1]) : null;
48      }
49
50      function parseAcceptLanguage(header) {
51        return (header || "")
52          .split(",")
53          .map((part) => {
54            const [tag, ...params] = part.trim().split(";");
55            const q = params.map((p) => p.trim()).find((p) => p.startsWith("q="));
56            return { tag, q: q ? Number(q.slice(2)) : 1 };
57          })
58          .filter((x) => x.tag)
59          .sort((a, b) => b.q - a.q)
60          .map((x) => x.tag);
61      }
62
63      // persisted (cookie) -> ?lang= -> Accept-Language -> en. Anything unsupported is
skipped.
64      function negotiateLang(req, searchParams) {
```

```
      const candidates = [
        cookieValue(req.headers.cookie, LANG_COOKIE),
        searchParams.get("lang"),
        ...parseAcceptLanguage(req.headers["accept-language"]),
      ];
      for (const c of candidates) {
        const b = baseLang(c);
        if (b && SITE_LOCALES.includes(b)) return b;
      }
      return "en";
    }

    const setHtmlLang = (html, lang) => html.replace(/<html lang="[^"]*"/i, `<html
lang="${lang}"`);

    async function toRequest(req) {
      const url = `http://${req.headers.host || "localhost"}${req.url}`;
      let body;
      if (req.method !== "GET" && req.method !== "HEAD") {
        const chunks = [];
        for await (const c of req) chunks.push(c);
        body = Buffer.concat(chunks);
      }
      return new Request(url, { method: req.method, headers: req.headers, body });
    }

    async function send(res, response) {
      res.statusCode = response.status;
      for (const [k, v] of response.headers) res.setHeader(k, v);
      res.end(Buffer.from(await response.arrayBuffer()));
    }

    async function serveStatic(pathname, req, searchParams, res) {
      const rel = pathname === "/" ? "index.html"
        : pathname === "/capture" ? "capture.html"
        : pathname === "/own-model" ? "own-model.html"
        : pathname === "/flyer" ? "flyer.html"
        : pathname === "/flyer-kn" ? "flyer-kn.html"
        : pathname === "/flyer-hi" ? "flyer-hi.html"
        : pathname.replace(/^\/+/, "");
      const file = normalize(join(PUBLIC, rel));
      if (!file.startsWith(PUBLIC)) { res.statusCode = 403; return res.end("forbidden"); }
      try {
        let data = await readFile(file);
        res.setHeader("content-type", TYPES[extname(file)] || "application/octet-stream");
        if (LOCALIZED_PAGES.has(rel)) {
          // Pre-resolve the locale and stamp <html lang> so the first paint is correct (the
client shim
          // refines it). Vary on Accept-Language so a shared cache keeps per-language
copies distinct.
          const lang = negotiateLang(req, searchParams);
          if (lang !== "en") data = Buffer.from(setHtmlLang(data.toString("utf8"), lang),
"utf8");
          res.setHeader("content-language", lang);
          res.setHeader("vary", "Accept-Language");
        }
        res.end(data);
      } catch {
        res.statusCode = 404;
        res.end("not found");
      }
    }

    createServer(async (req, res) => {
      try {
```

```
126         const parsed = new URL(`http://x${req.url}`);
127         const pathname = decodeURIComponent(parsed.pathname);
128         if (pathname === "/api/request-access") {
129           if (req.method !== "POST") { res.statusCode = 405; return
res.end('{"ok":false,"error":"method not allowed"}'); }
130           return await send(res, await handleRequest(await toRequest(req), store, notify));
131         }
132         return await serveStatic(pathname, req, parsed.searchParams, res);
133       } catch (err) {
134         res.statusCode = 500;
135         res.end('{"ok":false,"error":"server error"}');
136         console.error(err);
137       }
138     }).listen(PORT, () => console.log(`khelsutra site listening on :${PORT}  (sqlite:
${DB_PATH})`));
139
```

16. user (2026-06-22T09:33:18.506Z)

```
1       -- D1 schema for Request-Access submissions.
2       -- Apply: npx wrangler d1 execute khelsutra-access --remote --file=schema.sql
3       CREATE TABLE IF NOT EXISTS access_requests (
4         id         INTEGER PRIMARY KEY AUTOINCREMENT,
5         created_at TEXT NOT NULL DEFAULT (datetime('now')),
6         segment    TEXT,            -- 'academy' | 'individual'
7         name       TEXT,
8         email      TEXT NOT NULL,
9         city       TEXT,
10        org_name   TEXT,            -- academy / club name
11        courts     TEXT,            -- '1' | '2-4' | '5+'
12        has_gpu    TEXT,            -- 'yes' | 'unsure' | 'no' | 'prefer-cloud'
13        storage    TEXT,            -- csv of gdrive,onedrive,dropbox,bucket,local,unsure
14        message    TEXT,            -- free text
15        consent    INTEGER DEFAULT 0,
16        country    TEXT,            -- coarse, from cf-ipcountry (no raw IP stored)
17        user_agent TEXT
18      );
19
```

17. assistant (2026-06-22T09:33:25.793Z)

```
Important finding: the doc's table (line 18-19) claims for the VM fallback:
- Entry: `server.js` (`node:http`) ? confirmed (line 8, `createServer`).
- Static: `server.js serves public/` ? confirmed (`PUBLIC`, `serveStatic`).
- Storage: `sqliteStore(db) ? node:sqlite` ? confirmed (`DatabaseSync`).
- Deploy: `node server.js under systemd` ? server runs via `node server.js`.

But note the doc's VM runbook (line 64) says **"Node 22+ only"** and server.js header (line 1)
also says "Node 22+ only". However the CI and provision script require **Node ?24** ("so
node:sqlite is unflagged"). `node:sqlite` was experimental/flagged before Node 22.5 and
stabilized later. This is an internal inconsistency in the doc itself (line 64 says Node 22+,
the provisioning section line 40 says ?24). Let me check the test files boot the real server,
and the provision/teardown scripts to confirm their described behavior, plus the `cf-ipcountry`
and `country` claims.
```

18. user (2026-06-22T09:33:26.746Z)

```
1       // [VM/droplet host harness] Boots the REAL server.js (node:http + node:sqlite) and
2       // round-trips it over HTTP ? the same proof we run on the droplet, automated. Keeps the
3       // repo compatible with BOTH hosts: src/index.test.js covers the Cloudflare (Worker+D1)
4       // path; this covers the Node+SQLite path. Run: node --test src/   (or: npm run test:vm)
5       import { test, before, after } from "node:test";
6       import assert from "node:assert/strict";
7       import { spawn } from "node:child_process";
```

```
8       import { createServer } from "node:net";
9       import { mkdtempSync, rmSync } from "node:fs";
10      import { tmpdir } from "node:os";
11      import { join } from "node:path";
12      import { DatabaseSync } from "node:sqlite";
13
14      function freePort() {
15        return new Promise((res) => {
16          const s = createServer();
17          s.listen(0, "127.0.0.1", () => { const p = s.address().port; s.close(() => res(p));
});
18        });
19      }
20      async function waitUp(url, ms = 10000) {
21        const end = Date.now() + ms;
22        while (Date.now() < end) {
23          try { await fetch(url); return; } catch { await new Promise((r) => setTimeout(r,
150)); }
24        }
25        throw new Error("server.js did not start listening in time");
26      }
27
28      let proc, base, dbPath, dir;
29
30      before(async () => {
31        const port = await freePort();
32        dir = mkdtempSync(join(tmpdir(), "ks-vm-"));
33        dbPath = join(dir, "access.db");
34        base = `http://127.0.0.1:${port}`;
35        const serverPath = join(import.meta.dirname, "..", "server.js");
36        proc = spawn(process.execPath, [serverPath], {
37          env: { ...process.env, PORT: String(port), DB_PATH: dbPath },
38          stdio: "ignore",
39        });
40        await waitUp(`${base}/`);
41      });
42
43      after(() => {
44        proc?.kill();
45        try { rmSync(dir, { recursive: true, force: true }); } catch {}
46      });
47
48      test("[VM host] server.js serves the form", async () => {
49        const html = await (await fetch(`${base}/`)).text();
50        assert.match(html, /id="af"/);
51        assert.match(html, /Get into the KhelSutra alpha/);
52      });
53
54      test("[VM host] serves /capture and /og.png", async () => {
55        assert.equal((await fetch(`${base}/capture`)).status, 200);
56        assert.equal((await fetch(`${base}/og.png`)).status, 200);
57      });
58
59      test("[VM host] serves the tool screencast as video/mp4 (plays in the post-signup
reveal)", async () => {
60        const res = await fetch(`${base}/video/tool-cuj.mp4`);
61        assert.equal(res.status, 200);
62        assert.equal(res.headers.get("content-type"), "video/mp4"); // not octet-stream ?
<video> can play it
63      });
64
65      test("[VM host] serves /own-model", async () => {
66        const res = await fetch(`${base}/own-model`);
67        assert.equal(res.status, 200);
68        assert.match(await res.text(), /Train a model that's yours/);
```

```
69      });
70
71      test("[VM host] serves the academy flyers (/flyer, /flyer-kn, /flyer-hi) as clean URLs",
async () => {
72        // Cloudflare maps these via asset html-handling; the VM host needs explicit routes so
the
73        // footer links work on BOTH hosts (parity).
74        for (const path of ["/flyer", "/flyer-kn", "/flyer-hi"]) {
75          const res = await fetch(`${base}${path}`);
76          assert.equal(res.status, 200, `${path} should serve 200 on the VM host`);
77          assert.match(res.headers.get("content-type"), /text\/html/, `${path} should serve
HTML`);
78          assert.match(await res.text(), /<html/i, `${path} should be an HTML page`);
79        }
80      });
81
82      test("[VM host] POST /api/request-access persists to SQLite", async () => {
83        const res = await fetch(`${base}/api/request-access`, {
84          method: "POST",
85          headers: { "content-type": "application/json" },
86          body: JSON.stringify({ segment: "academy", email: "harness@khelsutra.guru", has_gpu:
"yes", storage: ["gdrive", "bucket"], consent: true }),
87        });
88        assert.equal(res.status, 200);
89        assert.deepEqual(await res.json(), { ok: true });
90
91        const db = new DatabaseSync(dbPath);
92        const row = db.prepare("SELECT email, segment, has_gpu, storage, consent FROM
access_requests").get();
93        assert.equal(row.email, "harness@khelsutra.guru");
94        assert.equal(row.segment, "academy");
95        assert.equal(row.has_gpu, "yes");
96        assert.equal(row.storage, "gdrive,bucket");
97        assert.equal(row.consent, 1);
98      });
99
100     test("[VM host] honeypot + bad email are rejected without persisting", async () => {
101       const bot = await fetch(`${base}/api/request-access`, {
102         method: "POST", headers: { "content-type": "application/json" },
103         body: JSON.stringify({ email: "x@y.co", company: "spam" }),
104       });
105       assert.equal(bot.status, 200); // dropped silently
106       const bad = await fetch(`${base}/api/request-access`, {
107         method: "POST", headers: { "content-type": "application/json" },
108         body: JSON.stringify({ email: "nope" }),
109       });
110       assert.equal(bad.status, 400);
111
112       const db = new DatabaseSync(dbPath);
113       const { n } = db.prepare("SELECT COUNT(*) AS n FROM access_requests").get();
114       assert.equal(n, 1); // only the valid submission above
115     });
116
117     test("[VM host] GET /api/request-access -> 405", async () => {
118       assert.equal((await fetch(`${base}/api/request-access`)).status, 405);
119     });
120
121     // --- §6.5 N8: static-site language negotiation (same order as the SPA)
--------------------------
122     test("[VM host · N8] a Kannada browser (Accept-Language: kn) gets the kn-rendered page",
async () => {
123       const res = await fetch(`${base}/`, { headers: { "accept-language": "kn-
IN,kn;q=0.9,en;q=0.5" } });
124       const html = await res.text();
125       assert.match(html, /<html lang="kn"/, "a kn browser must auto-get <html
```

```
lang=\"kn\">");
126       assert.equal(res.headers.get("content-language"), "kn");
127       assert.match(res.headers.get("vary") || "", /accept-language/i, "must Vary on Accept-
Language for caches");
128       // The negotiated page still ships the client shim + switcher so the browser can
refine from here.
129       assert.match(html, /src="\/i18n\.js"/, "the negotiated page must still load the client
shim");
130       assert.match(html, /<select data-langsw|<option value="kn"/, "the switcher must be
present on the negotiated page");
131     });
132
133     test("[VM host · N8] ?lang= overrides the browser language", async () => {
134       const res = await fetch(`${base}/?lang=hi`, { headers: { "accept-language": "en-US" }
});
135       assert.match(await res.text(), /<html lang="hi"/, "?lang=hi must win over an en
Accept-Language");
136     });
137
138     test("[VM host · N8] the persisted cookie beats Accept-Language (returning user keeps
their choice)", async () => {
139       const res = await fetch(`${base}/capture`, { headers: { "accept-language": "kn",
cookie: "khelsutra.lang=hi" } });
140       assert.match(await res.text(), /<html lang="hi"/, "the persisted-pref cookie must win
over Accept-Language");
141     });
142
143     test("[VM host · N8] unsupported/absent language falls back to en (never a locale we
can't serve)", async () => {
144       const ta = await fetch(`${base}/?lang=ta`, { headers: { "accept-language": "ta-IN" }
});
145       assert.match(await ta.text(), /<html lang="en"/, "Tamil (unsupported) must fall back
to en");
146       const none = await fetch(`${base}/own-model`);
147       assert.match(await none.text(), /<html lang="en"/, "no signal -> en default");
148     });
149
150     test("[VM host · N8] non-localized assets are NOT rewritten (only the 3 marketing pages
negotiate)", async () => {
151       // The flyers are fixed-language pages ? negotiation must not touch them.
152       const res = await fetch(`${base}/flyer-kn`, { headers: { "accept-language": "hi" } });
153       assert.match(await res.text(), /<html lang="kn"/, "the Kannada flyer must stay kn
regardless of Accept-Language");
154     });
155
156     test("[VM host] serves the shared catalogs the shim fetches (/locales/*/common.json as
JSON)", async () => {
157       const res = await fetch(`${base}/locales/en/common.json`);
158       assert.equal(res.status, 200);
159       assert.match(res.headers.get("content-type") || "", /application\/json/, "catalogs
must serve as JSON");
160       const cat = await res.json();
161       assert.ok(cat.site && cat.lang, "the served catalog must carry the shared site.* +
lang.* keys");
162     });
163
```

19. user (2026-06-22T09:33:27.847Z)

```
#!/usr/bin/env bash
```

First-class, PROVIDER-AGNOSTIC provisioning for the KhelSutra site (Node + SQLite

Run from your LOCAL machine against a FRESH Ubuntu VM (DigitalOcean / GCE / any

it pushes site/ via `git archive` (PRIVATE-REPO-SAFE ? no clone, token, or secret on t

VM), installs Node, installs + starts a systemd service, opens the firewall, and VERIFIE it is actually serving. Idempotent (safe to re-run). Pairs with teardown-vm.sh so a destroy ? resume-on-a-different-provider cycle is clean and reproducible.

#

SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \ [REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] [NODE_MAJOR=24] [RESTORE_DB= site/scripts/provision-vm.sh

#

NODE_MAJOR must be >= 24 so `node:sqlite` is unflagged (server.js imports it directl

```
set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
PORT="${PORT:-8787}"
NODE_MAJOR="${NODE_MAJOR:-24}"
SITE="$(cd "$(dirname "$0")/.." && pwd)"          # the site/ dir (script lives in
site/scripts/)
REPO="$(cd "$SITE/.." && pwd)"                      # repo root (for git archive)
SSHO=(-i "$KEY" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=10)

echo "? [1/5] waiting for SSH at $SSH_TARGET ..."
for _ in $(seq 1 60); do ssh "${SSHO[@]}" "$SSH_TARGET" true 2>/dev/null && break || sleep 5;
done
ssh "${SSHO[@]}" "$SSH_TARGET" true                 # final attempt ? fail loud if still down

echo "? [2/5] pushing site/ via git archive (private-repo-safe; no secret on the VM) ..."
ssh "${SSHO[@]}" "$SSH_TARGET" "rm -rf '$DIR/site' && mkdir -p '$DIR/site'"
```

core.autocrlf=false forces LF in the archive even when the operator runs on Windows

```
=====TEARDOWN=====
#!/usr/bin/env bash
```

Clean teardown for the destroy ? resume-anywhere cycle. EXPORTS the SQLite state local timestamped backup BEFORE you destroy the VM, so registrations survive the r different provider. Then stops the service and (optionally) destroys the DO droplet. Resume on any fresh VM with: RESTORE_DB=<the backup> SSH_TARGET=root@NE

#

SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] [REMOTE_DIR=/sr [BACKUP_DIR=./vm-state-backups] [DESTROY=1 DROPLET_ID=12345678] site/scrip

```
set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
BACKUP_DIR="${BACKUP_DIR:-./vm-state-backups}"
SSHO=(-i "$KEY" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=10)
mkdir -p "$BACKUP_DIR"

STAMP="$(ssh "${SSHO[@]}" "$SSH_TARGET" 'date -u +%Y%m%dT%H%M%SZ')"
OUT="$BACKUP_DIR/access-$STAMP.db"
echo "? [1/3] exporting state $SSH_TARGET:$DIR/access.db ? $OUT ..."
if ssh "${SSHO[@]}" "$SSH_TARGET" "test -f '$DIR/access.db'"; then
  scp "${SSHO[@]}" -q "$SSH_TARGET:$DIR/access.db" "$OUT"
  echo "   saved $(wc -c < "$OUT" | tr -d ' ') bytes ? resume with: RESTORE_DB=$OUT ? provision-
vm.sh"
else
  echo "   no access.db on the VM (nothing to export)"
fi
```

```
echo "? [2/3] stopping the service ..."
ssh "${SSHO[@]}" "$SSH_TARGET" "systemctl stop khelsutra-site 2>/dev/null || true"

if [ "${DESTROY:-}" = "1" ] && [ -n "${DROPLET_ID:-}" ]; then
```

20. assistant (2026-06-22T09:33:36.538Z)

```
The provision/teardown scripts confirm all the doc's claims about them: `git archive` (private-
repo-safe), Node ?24, systemd, firewall, verify; teardown exports DB before destroy, optional
`DESTROY=1 DROPLET_ID=` via doctl, `RESTORE_DB` resume. The smoke script and `cf-
ipcountry`/country claims are confirmed in code.

Now let me verify a couple of remaining specifics: the `droplet-smoke.sh` `SSH_TARGET` usage,
that `index.test.js` covers the CF path, and run the test suite to confirm both paths actually
pass (cheap and validates the "enforced" claim). Let me also confirm the `notify` injection ?
the doc's `handleRequest(request, store)` signature claim says "takes the store by injection"
but the actual signature is now `handleRequest(request, store, notify)` (3 params). That's a
minor evolution. Let me run the tests.
```

21. user (2026-06-22T09:33:38.156Z)

```
#!/usr/bin/env bash
# Live VM smoke test: copy site/ to a droplet, boot server.js, round-trip it, report.
# Proves the repo's Node + SQLite path on a REAL VM (the Cloudflare path is covered b
# `npm test`). Idempotent; cleans up its temp DB. Needs Node >= 22 on the VM (or a
# self-contained binary at $REMOTE_DIR/node/bin/node).
#
#   SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \
#     [REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] scripts/droplet-smoke.sh
set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
PORT="${PORT:-8787}"
SITE="$(cd "$(dirname "$0")/.." && pwd)" # the site/ folder (script lives in site/scripts/)
SSHO=(-i "$KEY" -o StrictHostKeyChecking=accept-new)

echo "? scp $SITE  ?  $SSH_TARGET:$DIR/site"
ssh "${SSHO[@]}" "$SSH_TARGET" "mkdir -p '$DIR'"
scp "${SSHO[@]}" -qr "$SITE" "$SSH_TARGET:$DIR/"
```

22. user (2026-06-22T09:33:41.060Z)

```
? [VM host] serves the tool screencast as video/mp4 (plays in the post-signup reveal) (3.7712ms)
? [VM host] serves /own-model (4.5173ms)
? [VM host] serves the academy flyers (/flyer, /flyer-kn, /flyer-hi) as clean URLs (15.289ms)
? [VM host] POST /api/request-access persists to SQLite (22.9292ms)
? [VM host] honeypot + bad email are rejected without persisting (7.4429ms)
? [VM host] GET /api/request-access -> 405 (1.2809ms)
? [VM host · N8] a Kannada browser (Accept-Language: kn) gets the kn-rendered page (3.913ms)
? [VM host · N8] ?lang= overrides the browser language (2.9446ms)
? [VM host · N8] the persisted cookie beats Accept-Language (returning user keeps their choice)
(3.3031ms)
? [VM host · N8] unsupported/absent language falls back to en (never a locale we can't serve)
(4.6002ms)
? [VM host · N8] non-localized assets are NOT rewritten (only the 3 marketing pages negotiate)
(2.4646ms)
? [VM host] serves the shared catalogs the shim fetches (/locales/*/common.json as JSON)
(2.4122ms)
? submit shows an obvious in-flight state (disabled + aria-busy + 'Sending?') (408.7033ms)
```

```
? on failure the button is restored ? never stuck on a frozen 'Sending?' (96.0841ms)
? after sign-up BOTH videos appear and a ? closes the reveal (69.5688ms)
? A2: after sign-up the reveal offers a WhatsApp share of the reel (wa.me, safe, localizable)
(48.7037ms)
? the auto-close HOLDS while the reel is playing (no mid-watch cut-off) (50.5625ms)
? tests 86
? suites 0
? pass 86
? fail 0
? cancelled 0
? skipped 0
? todo 0
? duration_ms 2348.2038
```

23. assistant (2026-06-22T09:33:48.766Z)

```
All 86 tests pass, both Cloudflare (D1) and VM (node:sqlite) paths verified. The `droplet-
smoke.sh` confirms `SSH_TARGET=root@host` usage (doc line 31 matches).

Now let me check the doc's claim about `node:sqlite` requiring Node 24 vs the runbook's "Node
22+". Let me verify the "Verified live (2026-06-20)" claims aren't independently falsifiable
from the repo (those are external/historical ? unverifiable from repo). Let me also confirm
`index.test.js` exists for the CF path and check whether the doc mentions `better-sqlite3` which
isn't in deps (it's an alternative, the code uses node:sqlite). Let me confirm there's no
`better-sqlite3` requirement issue, and check the `public/` files cited indirectly (og.png
served). Already confirmed `/og.png` serves 200 in tests.

Let me do a final check on the parity workflow mentioned (CI runs on every PR) and confirm
`index.test.js` content references the CF/D1 path.
```

24. user (2026-06-22T09:33:50.492Z)

```
// Tests for the Request-Access logic + the storage adapters. Run: node --test src/
import { test } from "node:test";
import assert from "node:assert/strict";
import { DatabaseSync } from "node:sqlite";
import worker, { validate, toRow, isBot, handleRequest } from "./index.js";
import { d1Store, sqliteStore, insertArgs, INSERT_SQL } from "./store.js";
import { buildSubmissionEmail } from "./notify.js";

// A store spy (runtime-agnostic) ? what handleRequest depends on.
function fakeStore() {
  const rows = [];
  return { rows, async insertAccessRequest(r) { rows.push(r); } };
---PARITY WF---
```

Live-host parity ? the site runs on TWO hosts (Cloudflare Worker + the DO droplet's Node/SQLite twin) that must serve byte-identical pages. The unit suite proves the code works on each host; it CANNOT see that the two LIVE URLs have drifted (e.g. a merge auto-deployed Cloudflare but the droplet wasn't re-synced from git). This runs the existing scripts/parity-check.sh against both live URLs on a schedule ? NOT on PRs, so it never blocks development; a failure surfaces real production drift to the owner.

```
name: live-parity

on:
  schedule:
    - cron: "17 6 * * *" # daily ~06:17 UTC
  workflow_dispatch: {} # and on demand
```

Avoid overlapping runs if a manual dispatch races the cron.

```
concurrency:
  group: live-parity
  cancel-in-progress: false

jobs:
  parity:
```

25. assistant (2026-06-22T09:34:01.470Z)

I have everything I need. `index.test.js` confirms the CF/D1 path (imports `validate, toRow, isBot, handleRequest`, `d1Store, sqliteStore`). The `test.yml` workflow runs on every push to master + every PR with `npm test` (which runs all paths via `node --test`).

Let me summarize the findings. The doc is overwhelmingly accurate and proven. The only real issues are minor:

1. **Doc line 25**: "`node --test src/` runs BOTH paths" ? the actual CI/test command is `npm test` ? `node --test` (no `src/` path arg). The `src/` form still works but isn't what CI runs. Minor/cosmetic ? the doc itself says CI runs them, and the very next line cites the right test files. Low severity.

2. **Internal inconsistency on Node version**: doc line 64 / server.js header say "Node 22+ only", but the provisioning section (line 40) and CI/scripts require **Node ?24** ("so node:sqlite is unflagged"). A reader following the runbook on Node 22 could hit a flagged/unavailable `node:sqlite`. Medium severity (a reader could act on the wrong minimum).

3. **`handleRequest(request, store)` signature** (line 11) ? actual signature is now `handleRequest(request, store, notify)` (3rd param added). Minor; the injection point described is still accurate. Low.

Everything else ? file paths, routes, adapters, schema, scripts, CI Node 24, test files booting the real server, both paths green ? verified correct. The "Verified live (2026-06-20)" droplet claims are historical/external and not falsifiable from the repo (unverifiable, but consistent with the shipped scripts).

This is a live, accurate design+proof doc ? not an archive candidate (unlike the multilingual tracker, this describes ongoing portability infrastructure that's all present in code).

26. user (2026-06-22T09:34:37.762Z)

Structured output provided successfully

27. assistant (2026-06-22T09:34:48.383Z)

I've completed the audit of `docs/PORTABILITY.md`. The doc is in good shape (minor-staleness). Verdict and key findings:

Overall: minor-staleness. Every cited file, route, adapter, and script exists and matches the code, and I ran the full site suite (86 tests, both the Cloudflare-D1 and the booted-server.js node:sqlite paths) ? all green. This is a live, accurate design+proof doc, not an archive candidate.

The only real blemish worth acting on:

- **Medium ? Node version inconsistency.** The VM runbook (`docs/PORTABILITY.md:64`) and `site/server.js:1` say **"Node 22+ only"**, but the doc's own provisioning section (line 40), `site/scripts/provision-vm.sh` (`NODE_MAJOR` default 24, "must be >= 24"), and `.github/workflows/test.yml:17` all require **Node >=24** so `node:sqlite` is unflagged. A reader following the runbook on Node 22 could hit a flagged/unavailable `node:sqlite`. Fix: make the runbook say "Node 24+".

Two low-severity nits:
- `docs/PORTABILITY.md:25` says "`node --test src/` runs BOTH paths" ? the command CI actually runs is `npm test` (`node --test`, no `src/` arg). Harmless.

- `docs/PORTABILITY.md:11` describes `handleRequest(request, store)`; the real signature is now `handleRequest(request, store, notify)`. The injection point it describes is still accurate.

The "Verified live (2026-06-20)" droplet run (lines 57-61) is a historical external event ? not falsifiable from the repo, so marked unverifiable, though it's fully consistent with the shipped scripts.

Relevant files inspected: `C:\Users\avidu\Projects\khelsutra\docs\PORTABILITY.md`, `...\site\src\index.js`, `...\site\src\store.js`, `...\site\server.js`, `...\site\schema.sql`, `...\site\src\server.test.js`, `...\site\src\index.test.js`, `...\site\scripts\provision-vm.sh`, `...\site\scripts\teardown-vm.sh`, `...\site\scripts\droplet-smoke.sh`, `...\.github\workflows\test.yml`, `...\.github\workflows\parity.yml`.